TLS/SSL Security Analysis with Attack + Defense

Perform TLS/SSL scans (SSLyze/testssl.sh). Demonstrate MITM attack and show HSTS , certificate pinning, and TLS 1.3 mitigate it.

Performed by:
MGBEMENA MMESOMACHUKWU CHUKWUEMEKA (MESO)
NIKITA NIAKHAI

Pipeline of the work

01 Setup & Configuration

Installing tools on VMs. **PKI**, generate **CA**

02 Configuring nginx

for both servers
Configuring **nginx, ssl, HSTS, TLS1.3, CA, proxy, domain name**

03 Testing server:8443

Executing **testssl** and **sslyze.** Analyzing

04 MITM

Setting up local hotspot for 3 nodes and configuring network settings, proxy. Intercepting and changing, sending.

05 HSTS & pinning

Demonstrating **HSTS** using 2 python **scripts**

06 TLS 1.3 mitigating

Showing how **TLS1.3** mitigates through configuration

01 - Setup and configuration

Installation of tools and VMs

- `openssl` - is installed by default on KALI
- `nginx` - is installed via sudo apt install
- `bettercup` - is installed via sudo apt install
- `Burp Suite` - is pre installed on KALI
- `ssl` - is installed using self signed certs.
- `testssl` - is installed via sudo apt install
- `sslyze` - is installed via sudo apt install

Setting up - Creating local CA and Server Certificate

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~]
└─$ mkdir -p ~/tls-demo/{certs,nginx,pcaps,mitm,clients}
```

1. Creating directory for artifacts

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ openssl genpkey -algorithm RSA -out rootCA.key -pkeyopt rsa_keygen_bits:4096
.....+.......+......+..+.+..+....+.............+......+....+..+..........+......
....+.....+.......+ ... +.....++++++++++++++++++++++++++++++++++++++++++++++* ... +....
+ ... +......+....+.........+......+..+....++++++++++++++++++++++++++++++++++++++++++++
```

2. Generating CA key

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 \
 -subj "/C=RU/ST=Tatarstan/L=Innopolis/O=ProjectCA/CN=Nikita Meso" \
 -out rootCA.pem
```

3. Creating certificate

Setting up the server (PKI) - 2

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ openssl genpkey -algorithm RSA -out server.key -pkeyopt rsa_keygen_bits:2048
...+.....+.+...+..++++++++++++++++++++++++++++++++++++++++*.+.........+.....+.+..+.
....+............+.........+.+..+...++++++++++++++++++++++++++++++++++++*.+..+.
.....++++++
```

4. Creating a server key

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ openssl req -new -key server.key -subj "/C=RU/ST=Tatarstan/L=Innopolis/O=Server
Lab/CN=server.lab" -out server.csr
```

5. Creating CSR (Certificate signing request)

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ cat > server_ext.cnf <<EOF
subjectAltName = DNS:server.lab,IP:10.0.2.15
basicConstraints = CA:FALSE
keyUsage = digitalSignature,keyEncipherment
extendedKeyUsage = serverAuth
EOF
```

6. Creating and extension file for SAN

Setting up the server (PKI) - 3

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ openssl x509 -req -in server.csr -CA rootCA.pem -CAkey rootCA.key -CAcreateseri
al \
  -out server.crt -days 825 -sha256 -extfile server_ext.cnf
Certificate request self-signature ok
subject=C=RU, ST=Tatarstan, L=Innopolis, O=ServerLab, CN=server.lab
```

7. Signing server certificate

2 - Configuring nginx (name = tls-lab.local)

Creating 2 servers: vulnerable:8443 and hardened:9443

Setting up - Nginx configuration

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ sudo apt update && sudo apt install -y nginx
[sudo] password for mgbemena-mmesomachukwu:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages
```

1. Installing nginx

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ sudo mkdir -p /etc/nginx/ssl

┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ sudo nano /etc/nginx/sites-available/tls-demo
```

2. Creating ssl directory and copying the generated server.crt and server.key certificates in there

Setting up 2 - Configuring nginx for vulnerable:8443

```
  GNU nano 8.4                                                    /etc
server {
    listen 8443 ssl;
    server_name tls-lab.local;

    root /var/www/tls-lab;
    index index.html;

    ssl_certificate /etc/ssl/tls-lab/tls-lab-fullchain.pem;
    ssl_certificate_key /etc/ssl/tls-lab/tls-lab.key;

    # Allow only TLS1.2 (older) to demonstrate weaker configuration
    ssl_protocols TLSv1.2;
    ssl_ciphers 'ECDHE-RSA-AES128-SHA256:ECDHE-RSA-AES256-SHA:HIGH:!aNULL';
    ssl_prefer_server_ciphers on;

    add_header X-Lab "vulnerable" always;

    location / { try_files $uri $uri/ =404; }
}
```

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ sudo ln -s /etc/nginx/sites-available/tls-demo /etc/nginx/sites-enabled/tls-d
o
```

3. Creating nginx configuration: nginx.conf + /etc/nginx/tls-lab.local(symlinked)

Setting up 3 - Configuring nginx for hardened:9443

```
# Hardened HTTPS vhost
server {
    listen 9443 ssl http2;                    # use modern separate directive
    server_name tls-lab.local;

    root /var/www/tls-lab;
    index index2.html;

    # Provide full chain to clients
    ssl_certificate /etc/ssl/tls-lab/tls-lab-fullchain.pem;
    ssl_certificate_key /etc/ssl/tls-lab/tls-lab.key;

    # SSL trusted certificate is required by nginx for OCSP stapling validation
    ssl_trusted_certificate /etc/ssl/tls-lab/lab-rootCA.crt;

    # Protocols (only TLS1.3 + TLS1.2)
    ssl_protocols TLSv1.3 TLSv1.2;
    ssl_prefer_server_ciphers off;

    # TLS1.2 ciphers (TLS1.3 ciphers are auto-negotiated by OpenSSL)
    ssl_ciphers 'ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384';

    # Session and security
    ssl_session_cache shared:SSL:10m;
    ssl_session_timeout 10m;
    ssl_session_tickets off;

    # OCSP stapling
    resolver 8.8.8.8 valid=300s;      # resolver to use for OCSP queries (choose a resolver reachable from server)
    resolver_timeout 5s;

    ssl_stapling on;
    ssl_stapling_verify on;

    add_header Strict-Transport-Security "max-age=31536000; includeSubDomains; preload" always;
    add_header X-Lab "hardened" always;

    location / { try_files $uri $uri/ =404; }
}
```

3. Creating nginx configuration: nginx.conf + /etc/nginx/tls-lab.local(symlinked)

Setting up 4 - Configuring nginx

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful

┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ sudo systemctl restart nginx
```

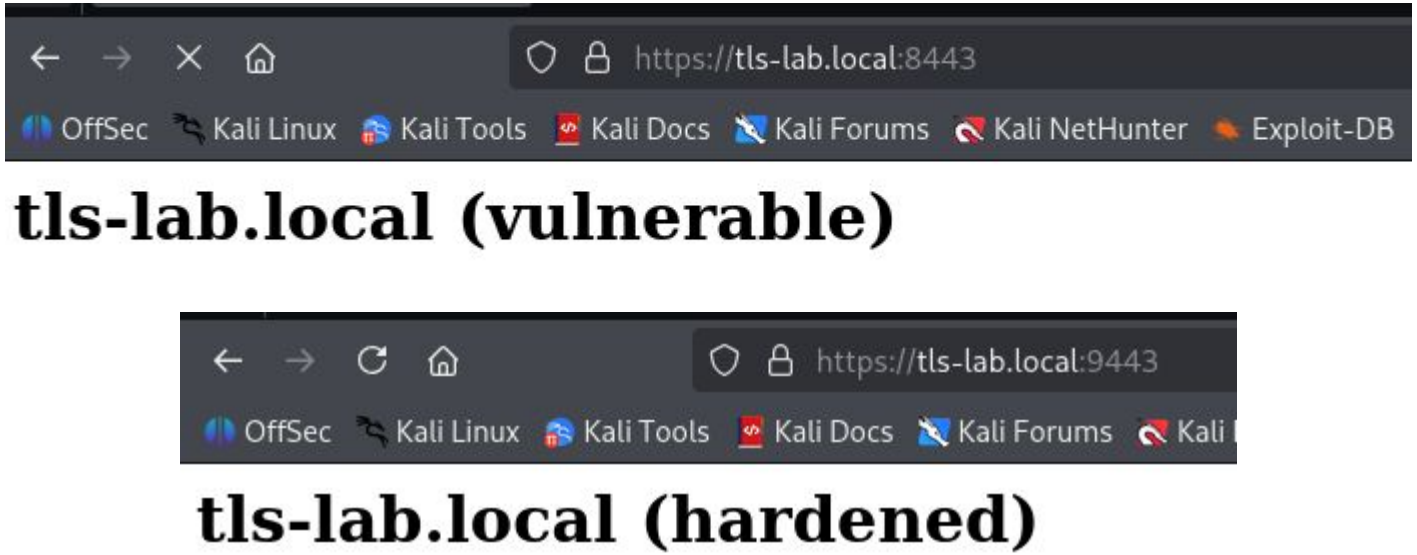
4. Testing the configuration with -t flag

   checks the configuration file syntax and then tries to open files referenced in the configuration file.

```
┌──(mgbemena-mmesomachukwu㉿kali)-[~/tls-demo/certs]
└─$ cat /etc/hosts
127.0.0.1       localhost
127.0.1.1       kali
::1             localhost ip6-localhost ip6-loopback
ff02::1         ip6-allnodes
ff02::2         ip6-allrouters

# Map the hostname to the IP address
10.0.2.15 server.lab
```

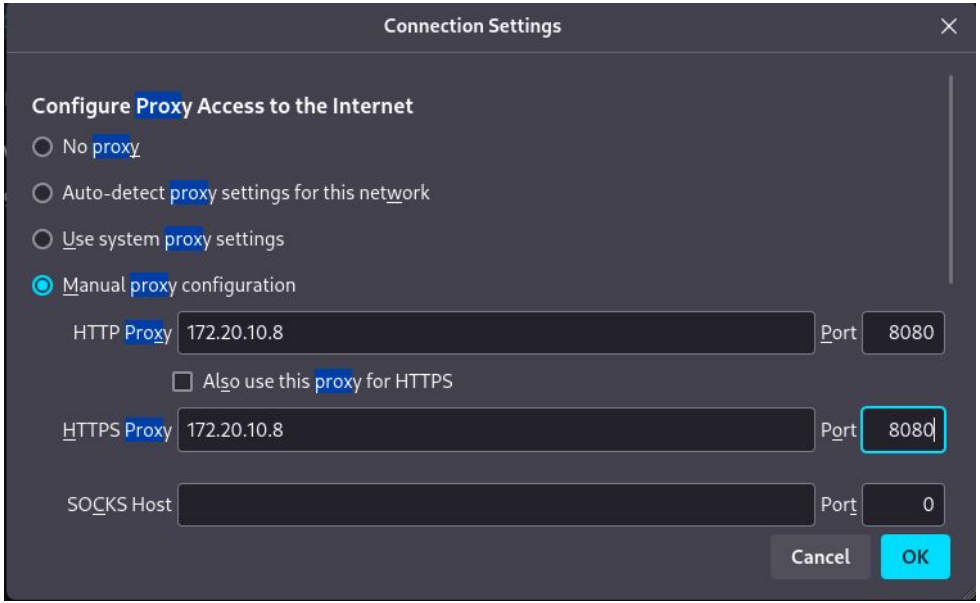
5. Mapping the hostname to the IP address

Setting up 5 - Configuring nginx



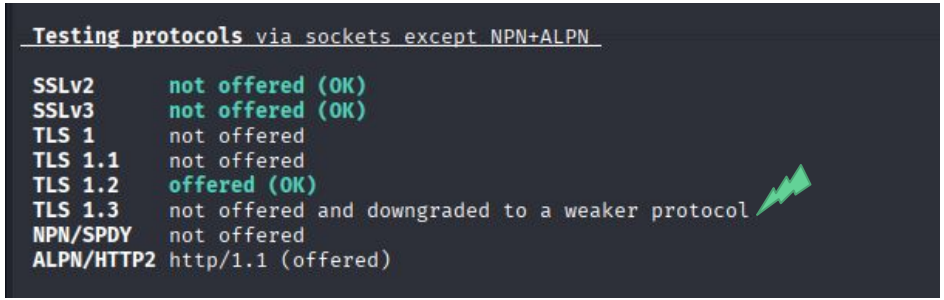
6. Servers are running.
   vulnerable:8443 and hardened:9443 servers for MITM

Setting up 6



7. Configuring proxy on Firefox Browser

3 - Testing certificates with testssl and sslyze for a vulnerable server :8443



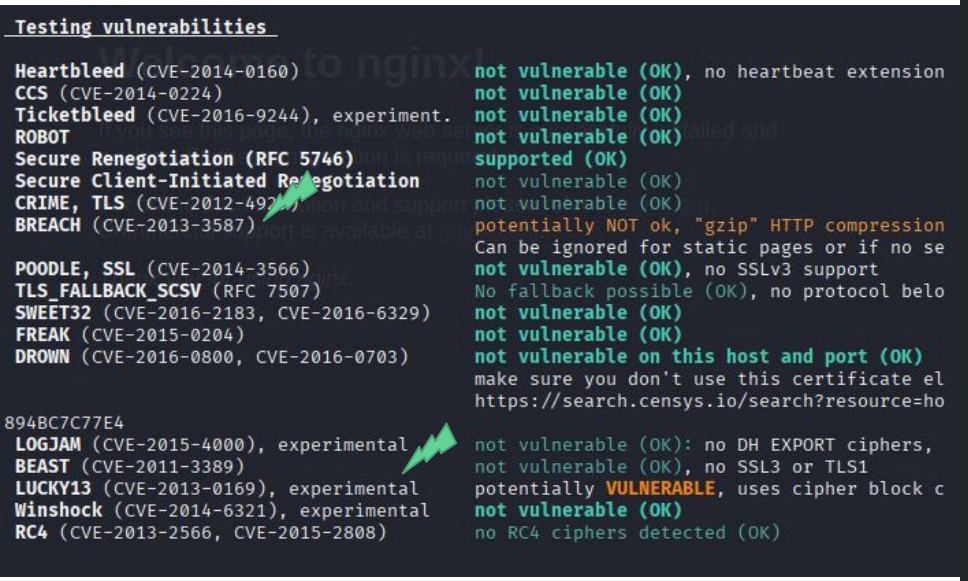
Executing testssl and showing the result.

Showing the protocol versions (1.0 , 1.1, 1.2, 1.3) supported.

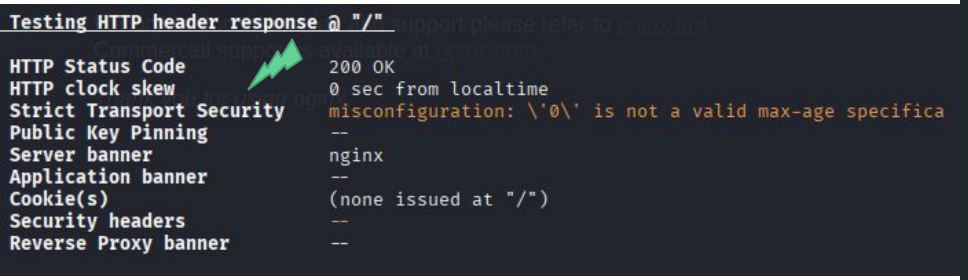
TLS 1.3 is not offered

```
Testing cipher categories

NULL ciphers (no encryption)                  not offered (OK)
Anonymous NULL Ciphers (no authentication)    not offered (OK)
Export ciphers (w/o ADH+NULL)                 not offered (OK)
LOW: 64 Bit + DES, RC[2,4], MD5 (w/o export)  not offered (OK)
Triple DES Ciphers / IDEA                     not offered
Obsoleted CBC ciphers (AES, ARIA etc.)        offered
Strong encryption (AEAD ciphers) with no FS   offered (OK)
Forward Secrecy strong encryption (AEAD ciphers)  offered (OK)
```

Testing for accepted cipher suite and whether weak ciphers such as RC4 are enabled

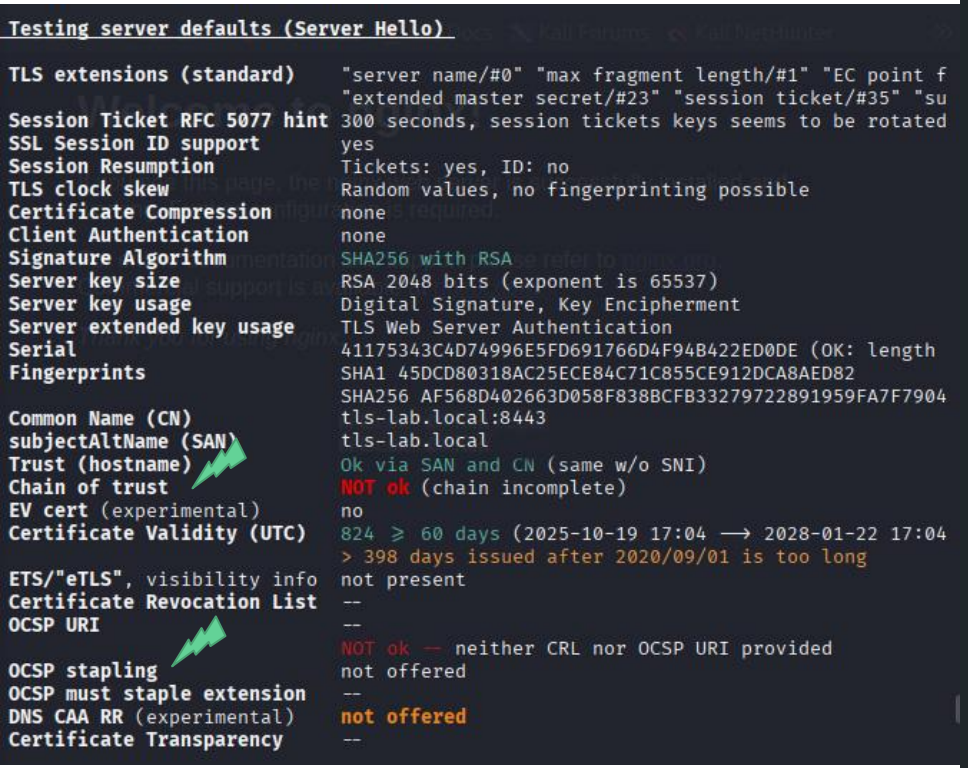


Scanning for potential vulnerabilities



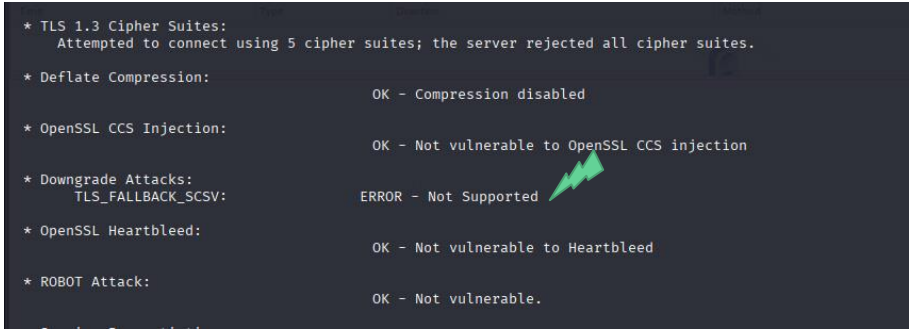
Note the misconfigured Strict Transport Security header (which can lead to a downgrade attack)

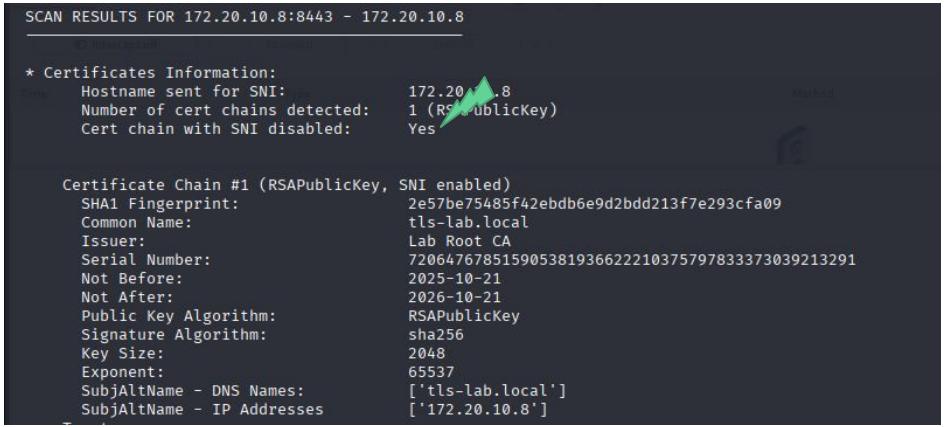
HSTS is not present



Checking for certificate chain issues such as expired chain, wrong chain or weak key size

Chain of trust is not ok, because of self-signed certs.

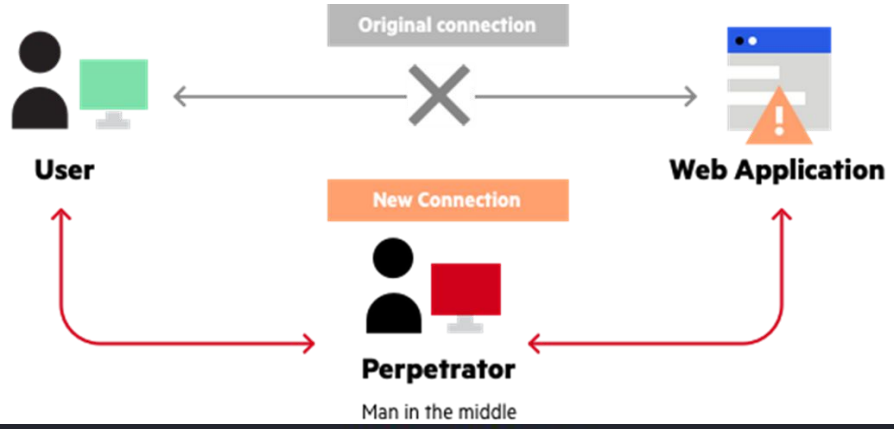


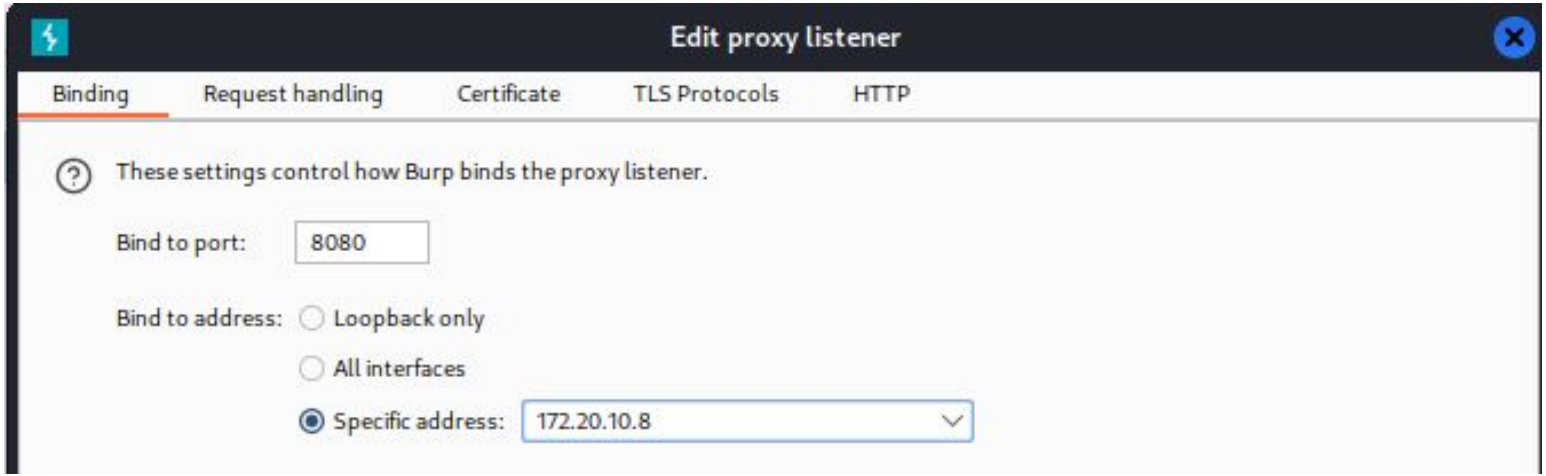


sslyze results

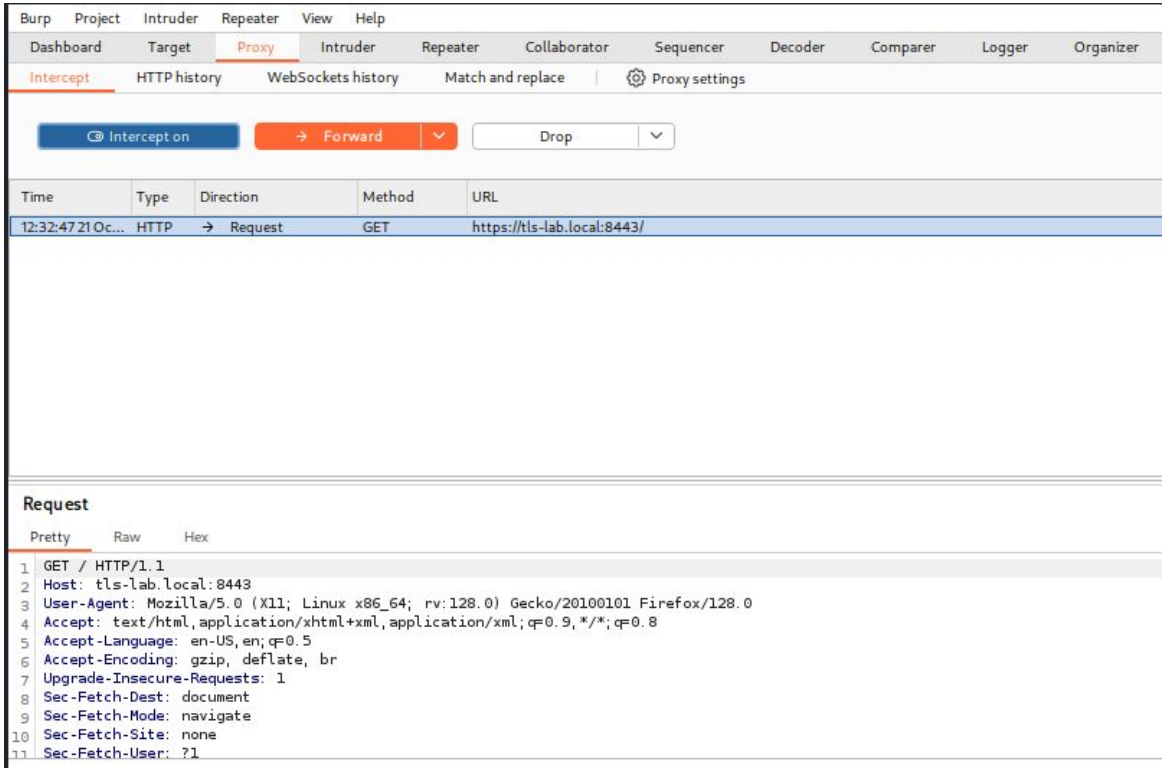
- server is vulnerable to desired attacks and TLS1.3 is not supported
- certificates

4 - Attack MITM

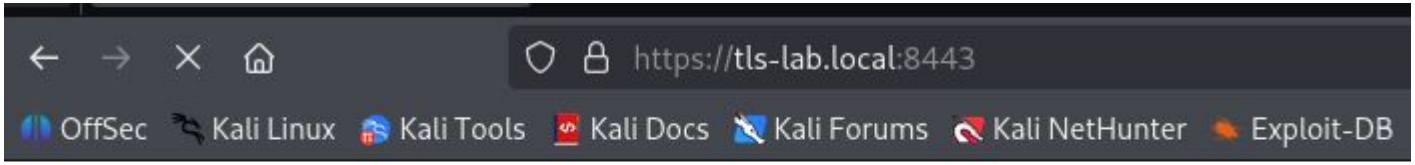




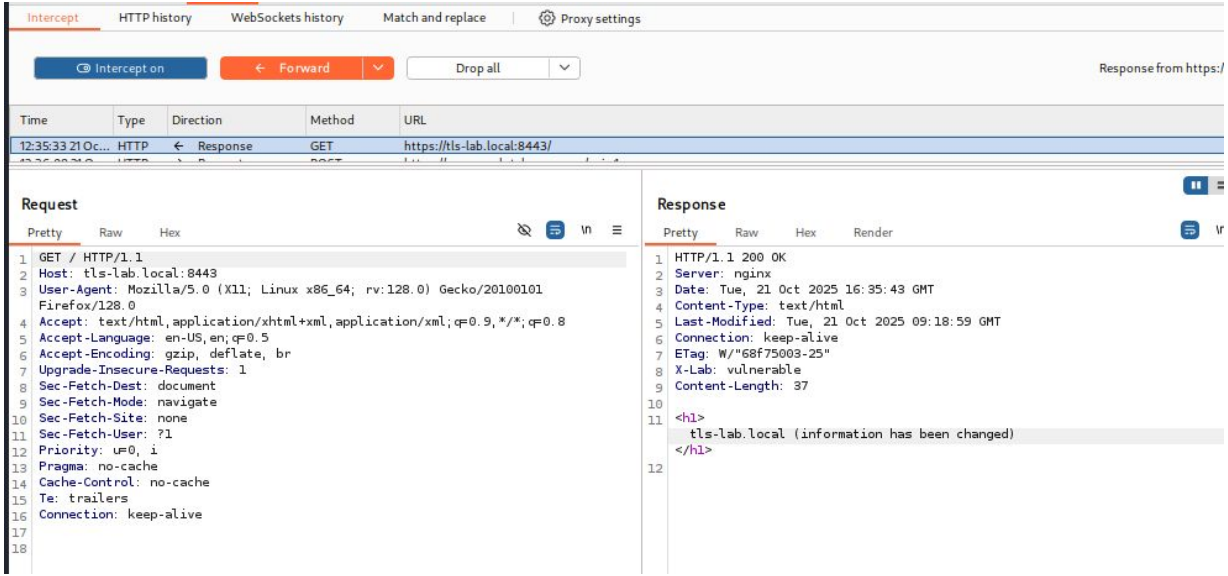
1. Configuring proxy on Burp Suite



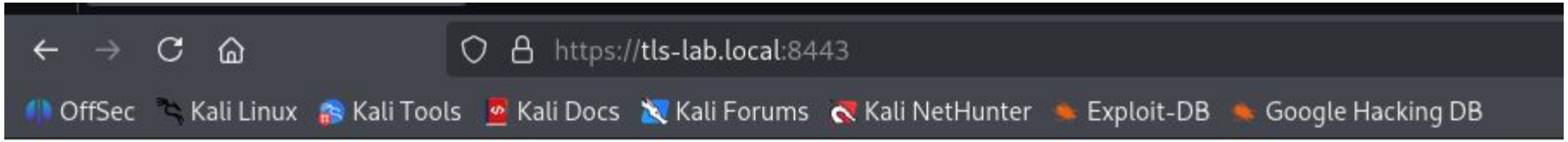
2. Request to a vulnerable server:8443



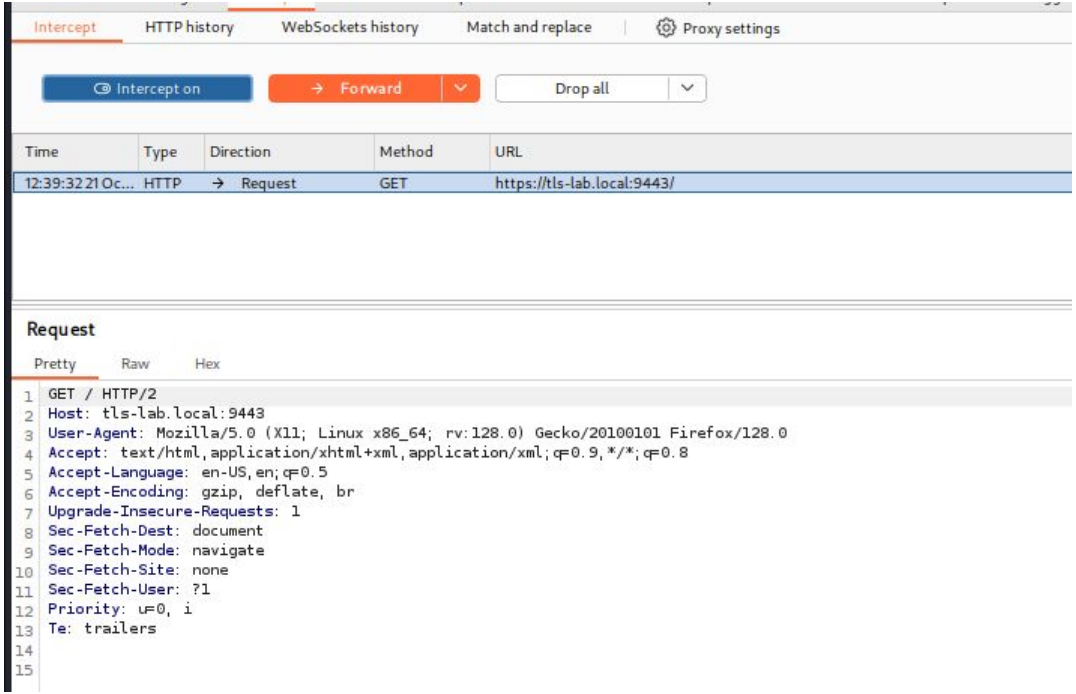
3. Normal response of vulnerable server:8443 to victim



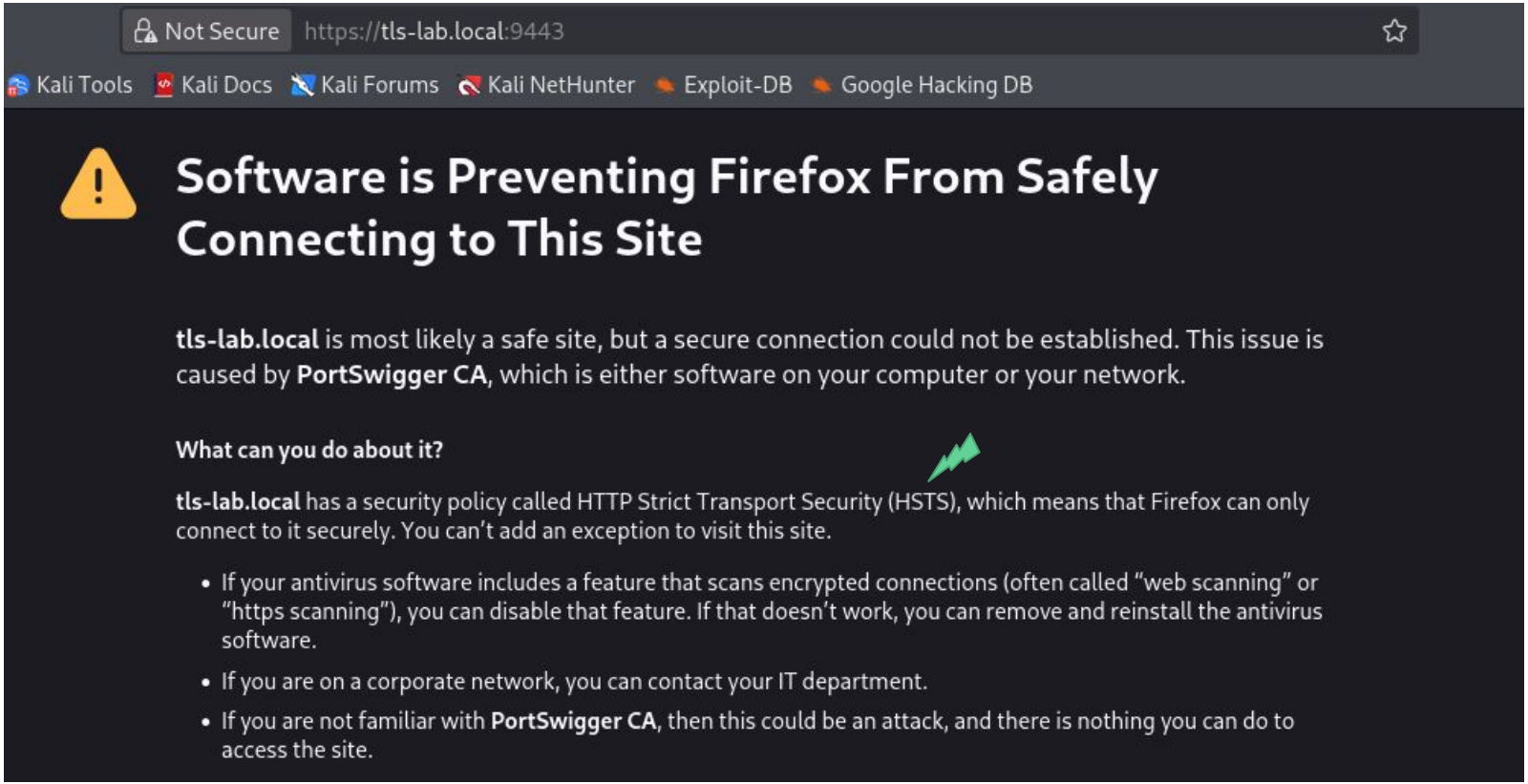
4. **Now** changing the info of a packet send to victim



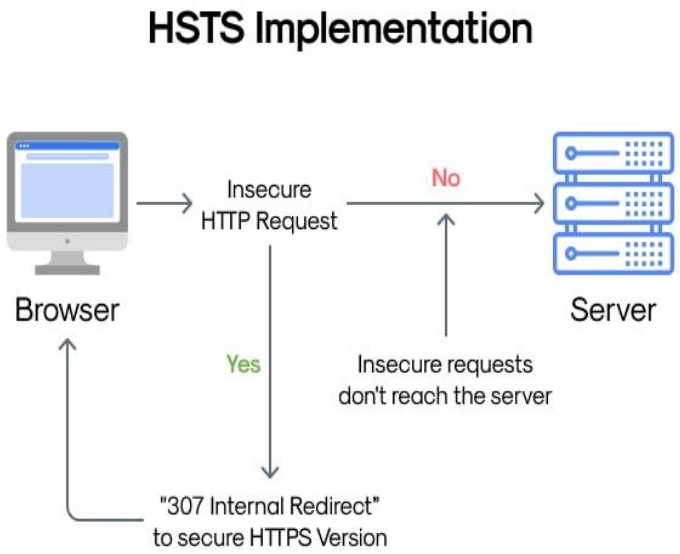
5. The packet was changed by attacker and victim receives it



6. Request for hardened:9443 server, after that no possibilities to manipulate and hijack, so that we say: hardening works



7. Victim visits hardened:9443. HSTS stops Burp Suite from downgrading the website

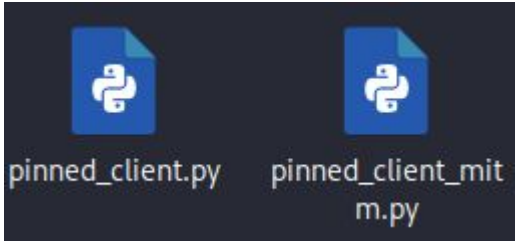


- defined a time window
- Browser refuses any HTTP-to-HTTPS downgrade
- Does **not** stop a MITM who can present a valid certificate the browser trusts (unless pinning is used).

5 - Certificate pinning

- using Python because HPKP (HTTP Public Key Pinning) is not supported by many browsers and client mngmt is necessary
- Implemented in Android apps

Scripts used



`pinned_client.py`

Normal connection

1. Open a TLS connection to HOST:PORT without verifying the certificate.
2. Download the server's X.509 certificate in DER format.
3. Use OpenSSL to extract the public key and compute its SHA256 hash.
4. If OpenSSL fails, fall back to hashing the entire certificate DER.
5. Compare the observed hash to EXPECTED_HEX; abort on mismatch (possible MITM).
6. If the pin matches, fetch the HTTPS page and print the first 1024 bytes.

`pinned_client_mitm.py`

MITM connection is impossible

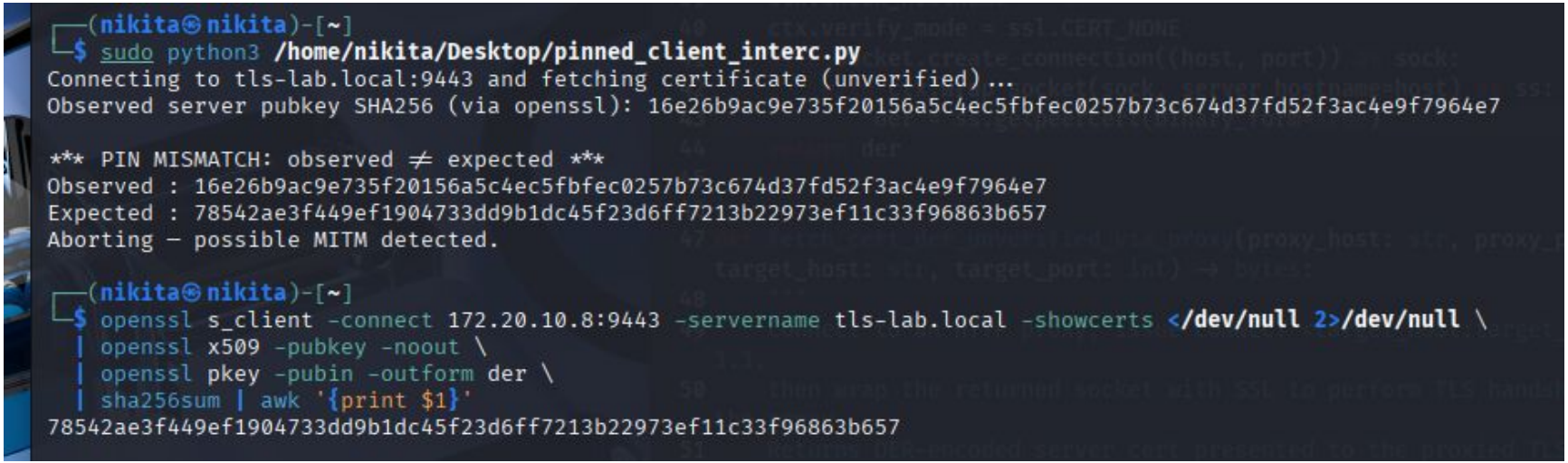
1. Issues HTTP CONNECT to proxy, then negotiates TLS through that proxy to target.
2. Retrieves the server's DER certificate from the proxied TLS handshake.
3. Uses OpenSSL to extract public-key DER and compute SHA256 fingerprint.
4. Fallback: if OpenSSL unavailable, hash whole certificate DER instead.
5. Compares observed hash to EXPECTED_HEX; mismatch indicates intercepting proxy or MITM.
6. If pin matches, performs HTTPS GET via proxy or direct and prints first 1024 bytes.

Certificate pinning, when there is no MITM

```
┌──(kali㉿kali)-[~/Desktop]
└─$ python3 cert_pinning.py
Connecting to tls-lab.local:9443 and fetching certificate (unverified)...
Observed server pubkey SHA256 (via openssl): 78542ae3f449ef1904733dd9b1dc45f23d6ff7213b22973ef11c33f96863b657

Pin matched — proceeding to fetch page (using unverified context for demo)...
HTTP status: 200
<h1>tls-lab.local (hardened) </h1>
```

`pinned_client.py`

Certificate pinning when there is MITM



MITM does not allowed when pinning

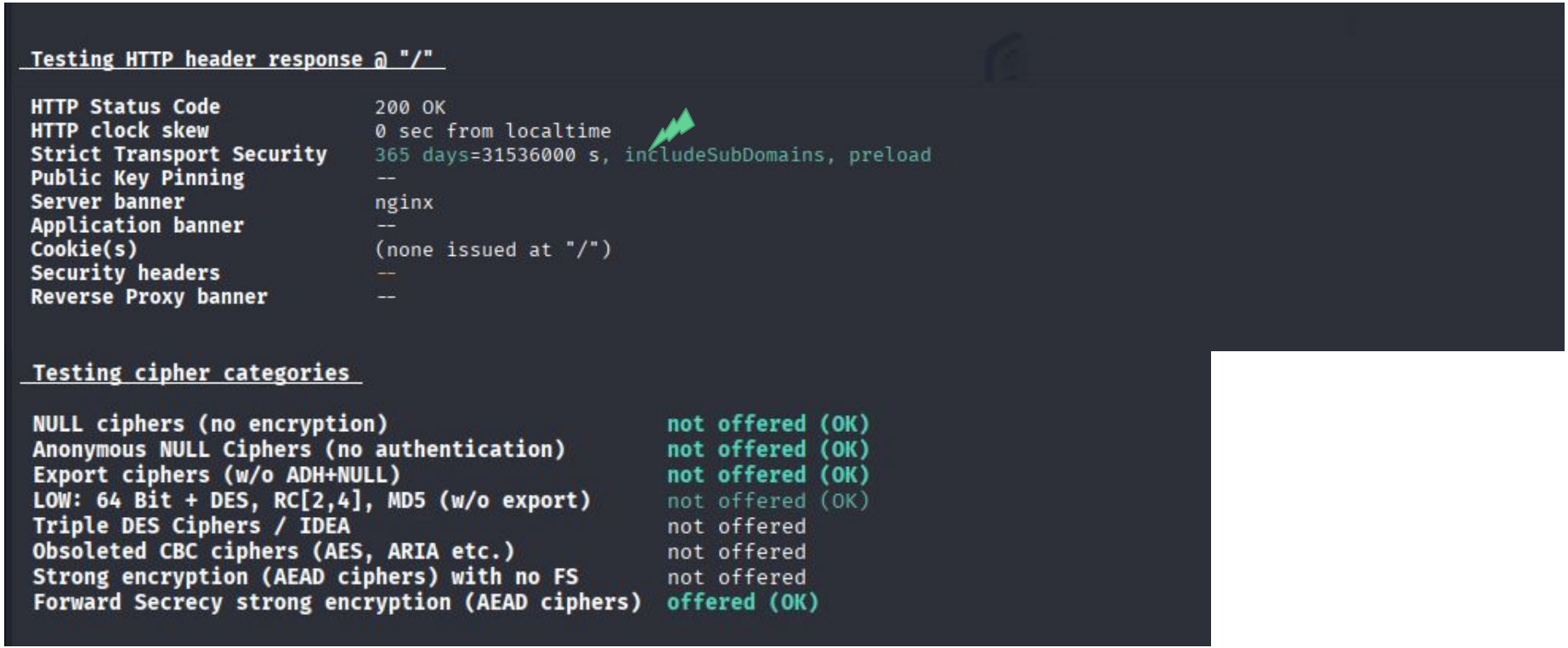
`pinned_client_mitm.py`

6 - TLS1.3 Mitigation

- **PFS** (Perfect Forward Security): long-term key exposure + ephemeral enc. key inside algorithms like Diffie Hellman
- **AEAD** (Authentication Encryption with Associated Data): confidentiality-integrity + preventing trivial modification. Binds processes of encryption and authentication.
- Shorter, **simpler handshake** reduces attack surface.

testssl and sslyze on hardened server: TLSv1.3.

HSTS is set

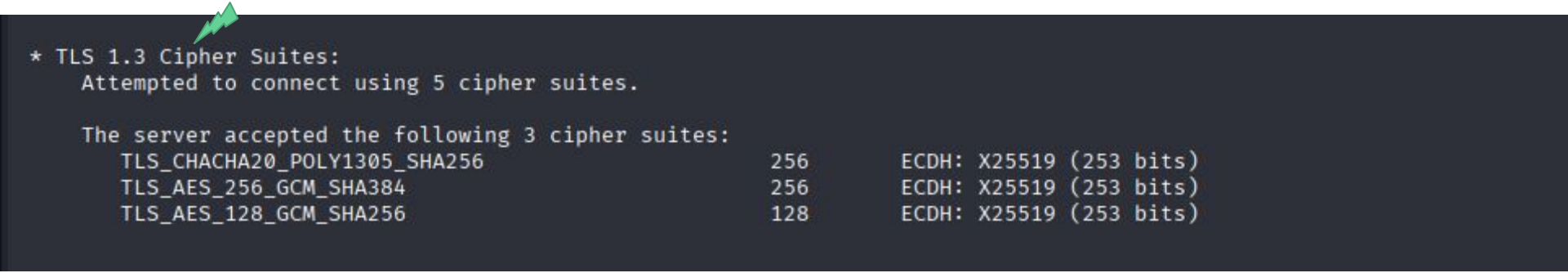


hardened server is not vulnerable for downgrade attacks and other attacks (LUCKY13)

```
* Deflate Compression:
                                     OK - Compression disabled

* OpenSSL CCS Injection:
                                     OK - Not vulnerable to OpenSSL CCS injection

* Downgrade Attacks:
      TLS_FALLBACK_SCSV:             OK - Supported

* OpenSSL Heartbleed:
                                     OK - Not vulnerable to Heartbleed

* ROBOT Attack:
                                     OK - Not vulnerable, RSA cipher suites not supported.
```

```
 Testing vulnerabilities

 Heartbleed (CVE-2014-0160)                not vulnerable (OK), no heartbeat extension
 CCS (CVE-2014-0224)                       not vulnerable (OK)
 Ticketbleed (CVE-2016-9244), experiment.  not vulnerable (OK), no session ticket extension
 ROBOT                                     Server does not support any cipher suites that use RSA key transport
 Secure Renegotiation (RFC 5746)           supported (OK)
 Secure Client-Initiated Renegotiation     not vulnerable (OK)
 CRIME, TLS (CVE-2012-4929)                not vulnerable (OK)
 BREACH (CVE-2013-3587)                    potentially NOT ok, "gzip" HTTP compression detected. - only supplied "/" tested
                                           Can be ignored for static pages or if no secrets in the page
 POODLE, SSL (CVE-2014-3566)               not vulnerable (OK), no SSLv3 support
 TLS_FALLBACK_SCSV (RFC 7507)              No fallback possible (OK), no protocol below TLS 1.2 offered
 SWEET32 (CVE-2016-2183, CVE-2016-6329)    not vulnerable (OK)
 FREAK (CVE-2015-0204)                     not vulnerable (OK)
 DROWN (CVE-2016-0800, CVE-2016-0703)      not vulnerable on this host and port (OK)
                                           make sure you don't use this certificate elsewhere with SSLv2 enabled services, see
                                           https://search.censys.io/search?resource=hosts&virtual_hosts=INCLUDE&q=D50F27B3530531C4FD39E5D19BFD6AB3ECFCED06B2341848E9C1
08E5D83A7D9F
 LOGJAM (CVE-2015-4000), experimental      not vulnerable (OK): no DH EXPORT ciphers, no DH key detected with <= TLS 1.2
 BEAST (CVE-2011-3389)                     not vulnerable (OK), no SSL3 or TLS1
 LUCKY13 (CVE-2013-0169), experimental     not vulnerable (OK)
 Winshock (CVE-2014-6321), experimental    not vulnerable (OK)
 RC4 (CVE-2013-2566, CVE-2015-2808)        no RC4 ciphers detected (OK)
```

Cipher suite: TLSv1.3



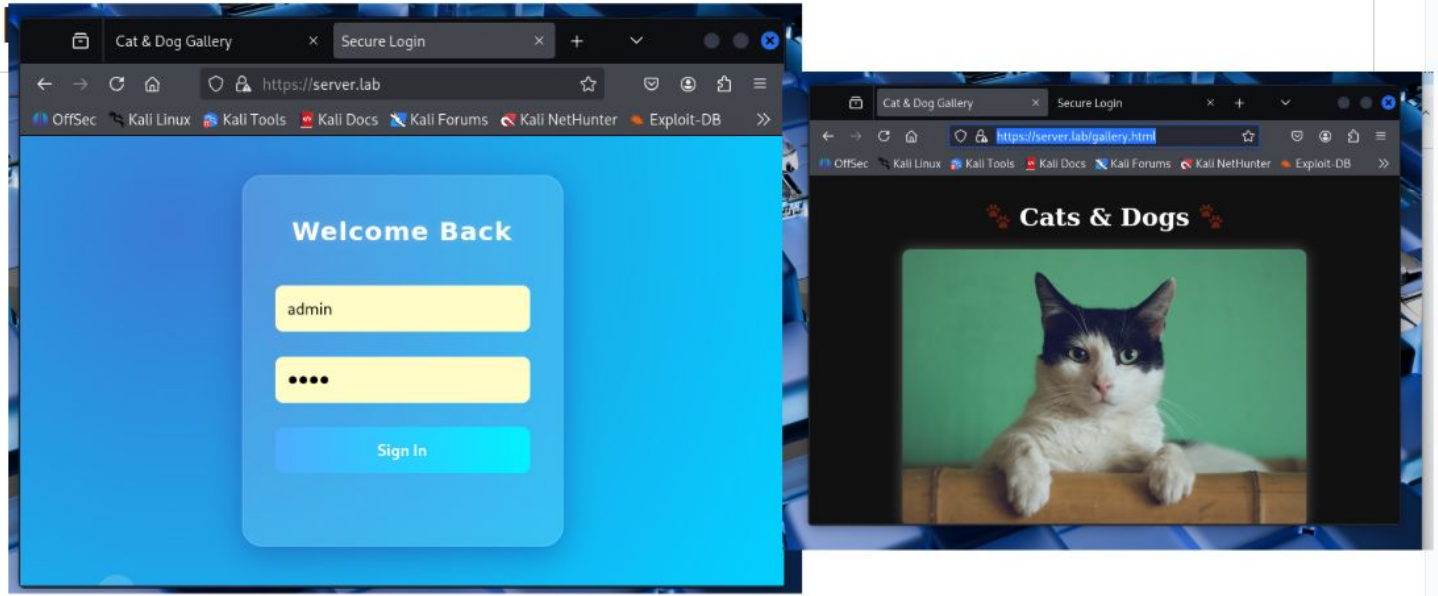
THANK YOU FOR THE ATTENTION

TLS/SSL Security Analysis with Attack + Defense

Perform TLS/SSL scans (SSLyze/testssl.sh), Demonstrate MITM attack and show HSTS , certificate pinning, and TLS 1.3 mitigate it.

Performed by:
MGBEMENA MMESOMACHUKWU CHUKWUEMEKA (MESO)
NIKITA NIAKHAI

First attempts: with bettercap and mtim



1. Creating UI with form (name and password) inside client server

```
  GNU nano 8.4                                spoof.cap
net.recon on
set arp.spoof.fullduplex true
set arp.spoof.targets 172.20.10.7
arp.spoof on
net.sniff on
```

```
┌──(nikita㉿nikita)-[~]
└─$ sudo bettercap -iface eth0 -caplet spoof.cap
bettercap v2.33.0 (built for linux amd64 with go1.22.6) [type 'help' for a li
st of commands]

[15:37:27] [sys.log] [war] Could not find mac for 172.20.10.1
[15:37:27] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.
[15:37:27] [sys.log] [war] arp.spoof full duplex spoofing enabled, if the rou
ter has ARP spoofing mechanisms, the attack will fail.
[15:37:27] [sys.log] [war] arp.spoof could not find spoof targets
[15:37:27] [endpoint.new] endpoint fe80::6a3d:12f0:d950:824 detected as 84:1b
:77:69:dc:82 (Intel Corporate).
```

```
172.20.10.0/28 > 172.20.10.9  » [13:57:49] [net.sniff.https] sni NIKITA.local > https://ecs.office.com
172.20.10.0/28 > 172.20.10.9  » [13:57:49] [net.sniff.https] sni NIKITA.local > https://ecs.office.com
172.20.10.0/28 > 172.20.10.9  » [13:58:04] [net.sniff.http.request] http NIKITA.local POST testphp.vulnweb.com/userinfo.php

POST /userinfo.php HTTP/1.1
Host: testphp.vulnweb.com
Content-Length: 21
Cache-Control: max-age=0
Origin: http://testphp.vulnweb.com
Content-Type: application/x-www-form-urlencoded
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 YaBrowser/25.8.0.0 Safari/537.3
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0
Referer: http://testphp.vulnweb.com/login.php
Connection: keep-alive
Dnt: 1
Upgrade-Insecure-Requests: 1
Accept-Encoding: gzip, deflate
Accept-Language: ru,en;q=0.9

uname=admin&pass=1234

172.20.10.0/28 > 172.20.10.9  » [13:58:05] [net.sniff.http.response] http 44.228.249.3:80 200 OK → NIKITA.local (5.5 kB text/html; charset=U
)
172.20.10.0/28 > 172.20.10.9  » [13:58:05] [net.sniff.http.response] http 44.228.249.3:80 302 Found → NIKITA.local (14 B text/html; charset=
8)
172.20.10.0/28 > 172.20.10.9  » [13:58:05] [net.sniff.http.request] http NIKITA.local GET testphp.vulnweb.com/login.php
```